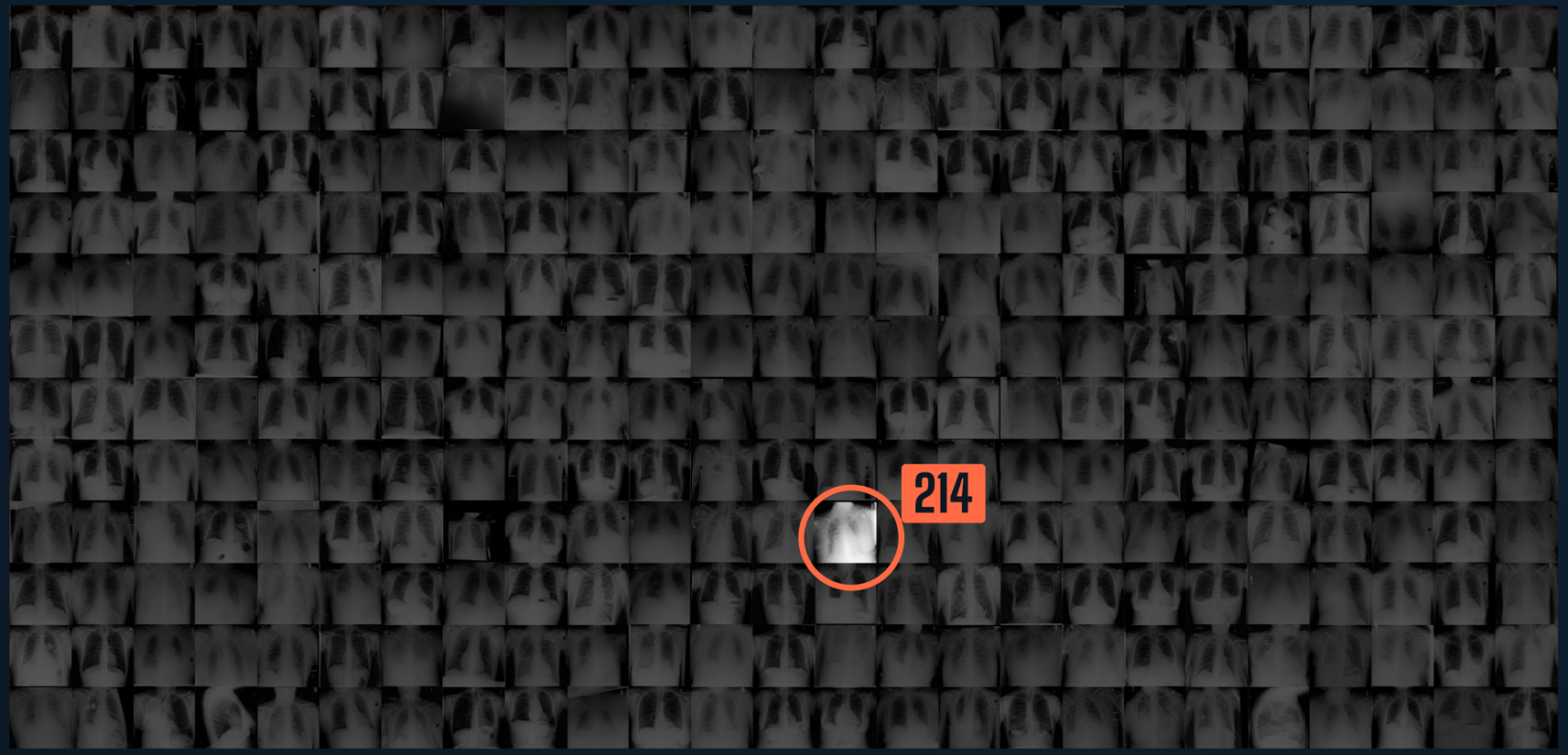
300 PLACAS. UN RADIÓLOGO.
POR ORDEN DE LLEGADA.



Posición 214

Doña Marta, 54 años.
Edema pulmonar.

A 3 min por placa, su radiografía se abre en 10 h 39 min.

Personaje ficticio. Placa real de NIH ChestX-ray14. Orden de llegada simulado; 3 minutos por placa es un supuesto.

895 987

estudios de rayos X en el IGSS en 2024: unos 2 455 al día. Solo Escuintla registra 5 981 al mes.

IGSS, Boletín estadístico de prestaciones en salud 2024, p. 56. Cifra de Escuintla: dato preliminar, 2026.

LOS DATOS: NIH CHESTX-RAY14, PÚBLICOS Y REALES

44 999
radiografías; el mínimo era 20 000

17,7 GB
en PNG de 1024 × 1024

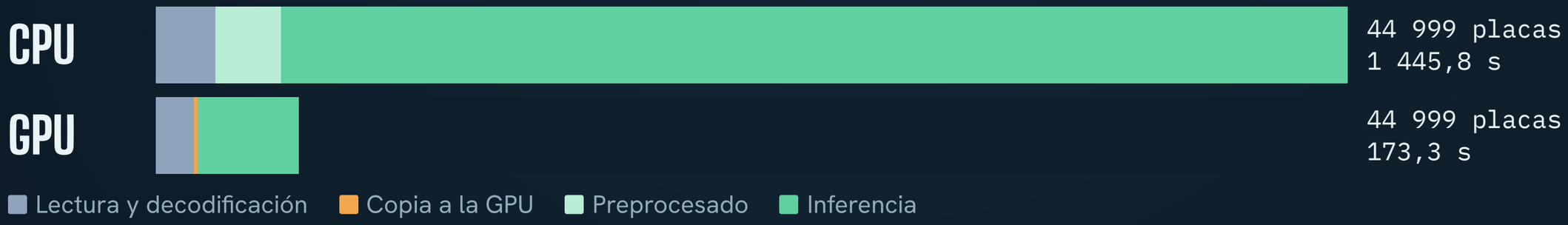
11 558
pacientes

421
PNG con alfa convertidos a gris; 0 ilegibles

nihcc.app.box.com/v/ChestXray-NIHCC · Wang et al. (2017) · Uso sin restricciones con cita al NIH Clinical Center

LA MISMA TAREA, DOS VECES

Leer PNG	Decodificar	Copiar en uint8	Recortar y reducir	DenseNet-121	Puntaje	Ordenar cola
CPU	CPU, hilos	PCIe	kernel propio	cuDNN, FP32	máx. de 5 hallazgos	GPU



8,3×

CPU 24,1 min, GPU 2,9 min. Copiar a la GPU: 3,96 s, el 2,3 %.



La barra de arriba es solo la GPU, a su propia escala: 173,3 s.

¿8,3× O 7,3×?

La GPU no decodifica: en las dos versiones lo hace la CPU. Lo que cambia es cuántos hilos de CPU quedan libres para hacerlo.

MEDIDO · CADA VERSIÓN EN SU MEJOR CONFIGURACIÓN

8,3×

$S = T_{CPU} / T_{GPU}$

$= 1\,445{,}8\ s / 173{,}3\ s$

$= 8{,}3\times$

CPU: la red usa los núcleos y decodifica con 8 hilos, 53,5 s. GPU: la red se va a la tarjeta y la CPU decodifica con 32 hilos, 27,6 s.

TEÓRICO · LEY DE AMDAHL

7,1×

$s = (19{,}1 + 53{,}5) / 1\,445{,}8 = 0{,}050$

$k = 1\,293{,}7 / 122{,}8 = 10{,}5$

$S = 1 / (s + (1 - s) / k)$

$= 1 / (0{,}050 + 0{,}950 / 10{,}5)$

$= 1 / 0{,}140 = 7{,}1\times$

s: lectura y decodificación, lo que no acelera la GPU. k: cuánto se acelera la inferencia. Supone que la parte de CPU no cambia.

MEDIDO · LOS MISMOS 8 HILOS EN LAS DOS

7,3×

$S = T_{CPU} / T_{GPU,\ 8\ hilos}$

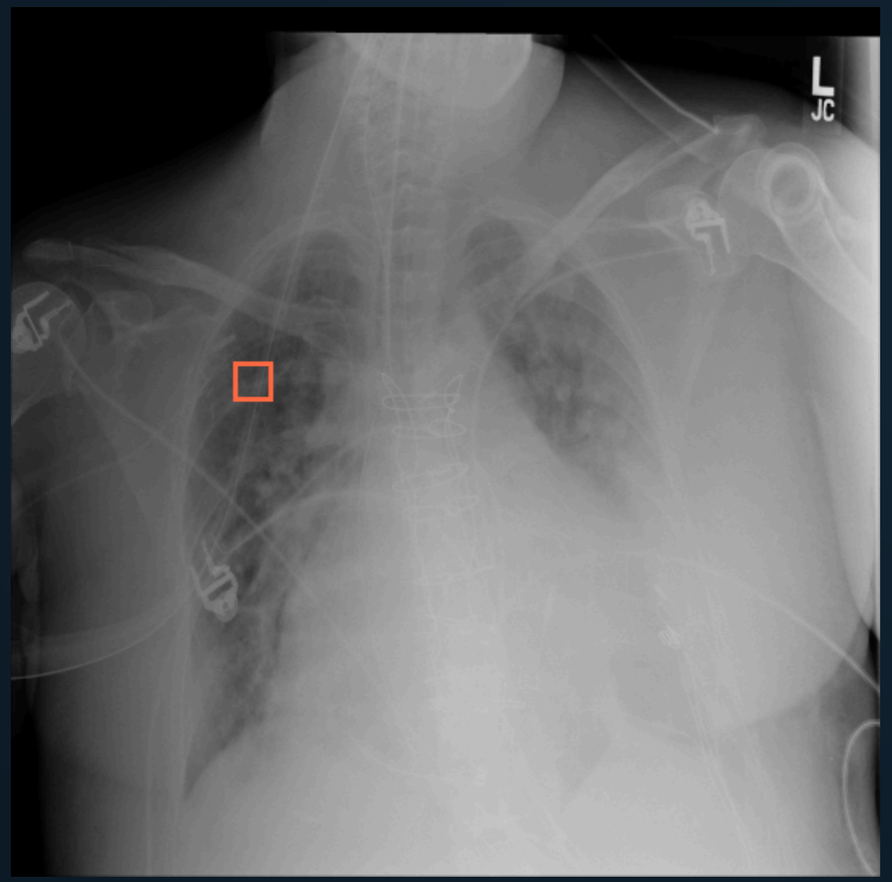
$= 1\,445{,}8\ s / 198{,}4\ s$

$= 7{,}3\times$

Decodificar vuelve a tardar 53,6 s, igual que en la CPU. Lo medido coincide con Amdahl.

Con 32 hilos, la versión GPU tarda 25 s menos (198,4 s → 173,3 s). Esos 25 s los gana la CPU al quedar libre de la red, no la GPU.

UN HILO CALCULA UN PÍXEL



66	67	67	66	72
64	66	69	69	73
65	67	69	71	75
68	69	70	72	77
71	72	74	77	81
73	75	77	79	83

hilo (62, 96)
bloque (3, 6)
caja 5 × 6 = 30 píxeles
suma 2 144
media 71,47
salida -450,0

219×

79,2 s en CPU, 0,36 s con el kernel

3,9×

menos copia: uint8 3,96 s, float32 15,33 s

50 176 hilos por placa · bloques de 16 × 16 · 3,2 millones de hilos por lote de 64 · pase el cursor sobre la placa

EL KERNEL, LÍNEA POR LÍNEA

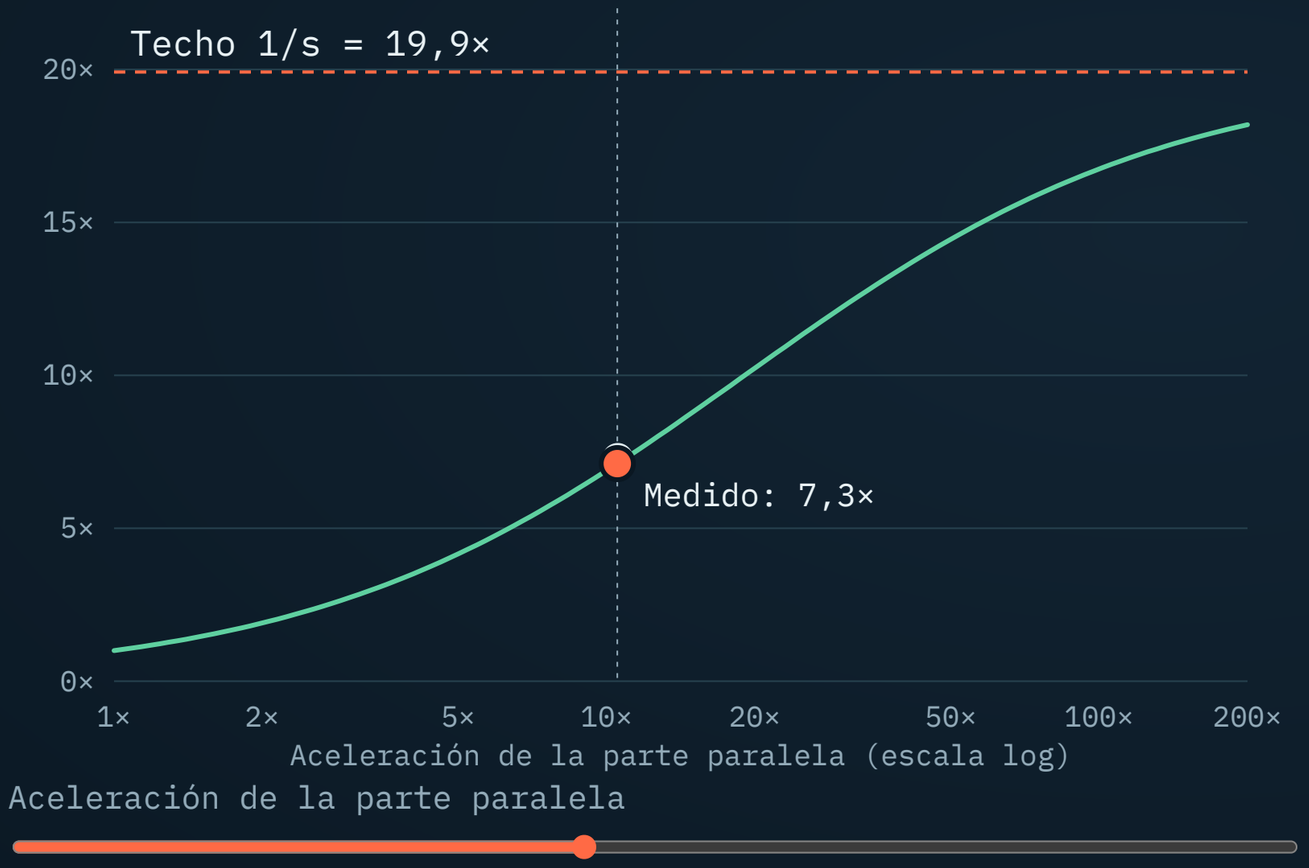
Resumido de gpu/preprocess.py · 14 × 14 × 64 bloques de 16 × 16 hilos = 3,2 millones de hilos por lote

```cpp
template <typename T>
__global__ void crop_area_normalize(const T* src, float* dst,
        const int* starts, const int* ends, int width, int out,
        float scale, float shift)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= out || y >= out) return;
    const T* img = src + blockIdx.z * width * width;
    int r0 = starts[y], r1 = ends[y];
    int c0 = starts[x], c1 = ends[x];
    float sum = 0.0f;
    for (int r = r0; r < r1; ++r)
        for (int c = c0; c < c1; ++c)
            sum += (float)img[r * width + c];
    float mean = sum / ((r1 - r0) * (c1 - c0));
    dst[(blockIdx.z * out + y) * out + x] =
        __fadd_rn(__fmul_rn(mean, scale), shift);
}
```

1. **Una plantilla para dos tipos.** El mismo kernel lee uint8 o float32. `__global__`: corre en la GPU y lo lanza la CPU.
2. **¿Qué píxel soy?** Cada hilo saca su columna y su fila de salida con su bloque y su posición dentro del bloque.
3. **¿Qué placa?** El eje z de la cuadrícula elige una de las 64 placas del lote.
4. **¿Dónde está mi caja?** Dos tablas calculadas una vez en la CPU dicen dónde empieza y termina: 5 o 6 píxeles por lado.
5. **Sumar.** Lee de 25 a 36 enteros de 1 byte y los pasa a float aquí, no antes de copiar.
6. **Promediar, normalizar y escribir.** media × 2048/255 − 1024, sin FMA para dar lo mismo que NumPy. Una escritura por hilo.

Ningún hilo espera a otro: 3,2 millones trabajan a la vez. CPU 79,2 s → GPU 0,36 s = 219×.

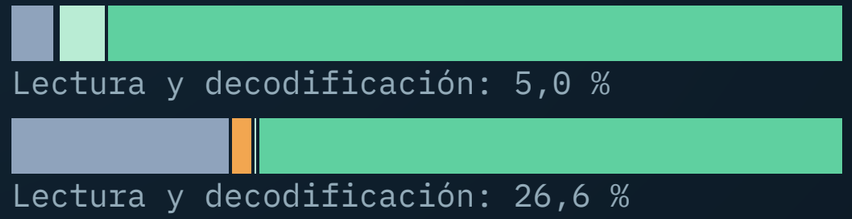
¿POR QUÉ NO 200 VECES MÁS RÁPIDO?



7,1×

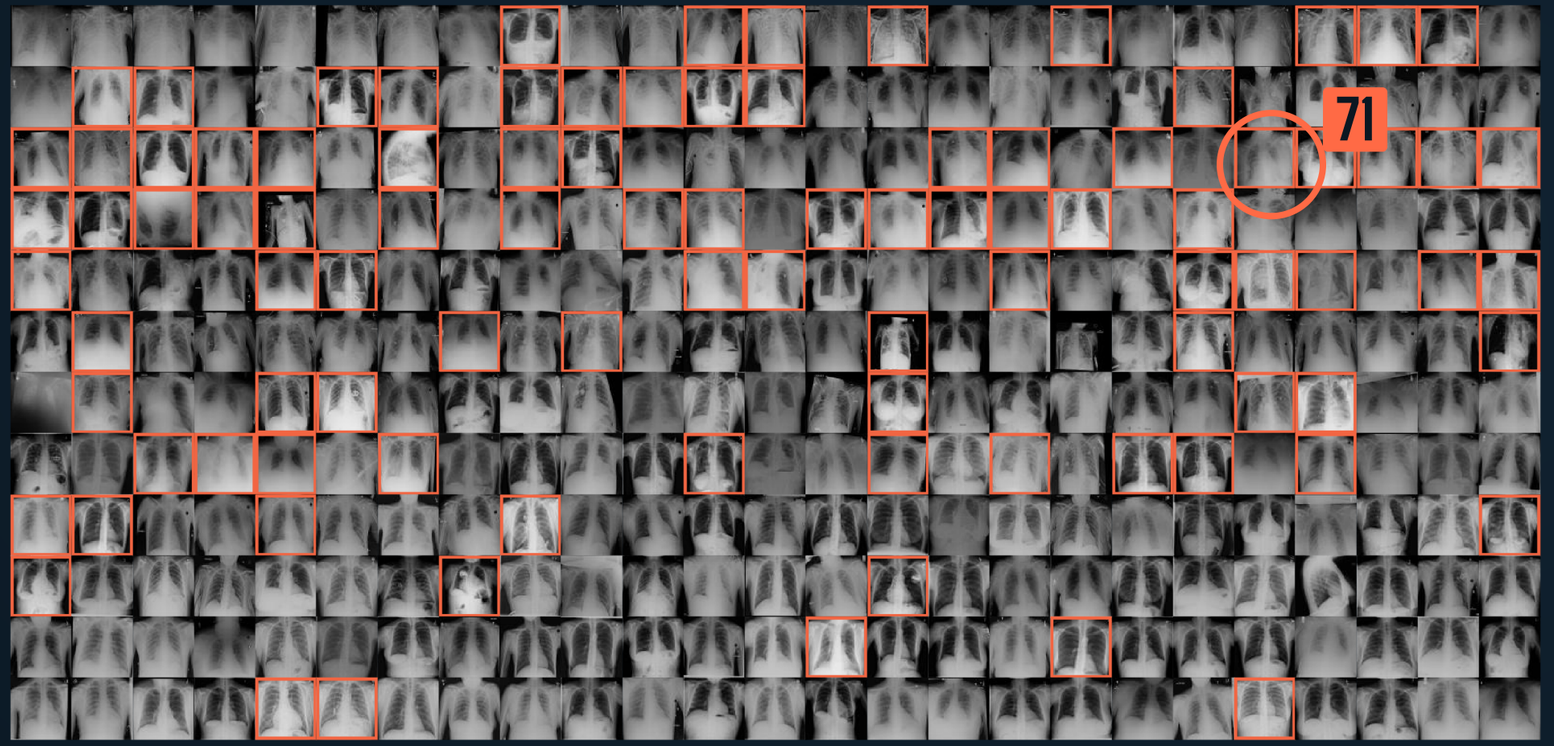
de speedup total si el resto fuera 10,5× más rápido. El 5,0 % sigue en la CPU.

El cuello de botella se movió



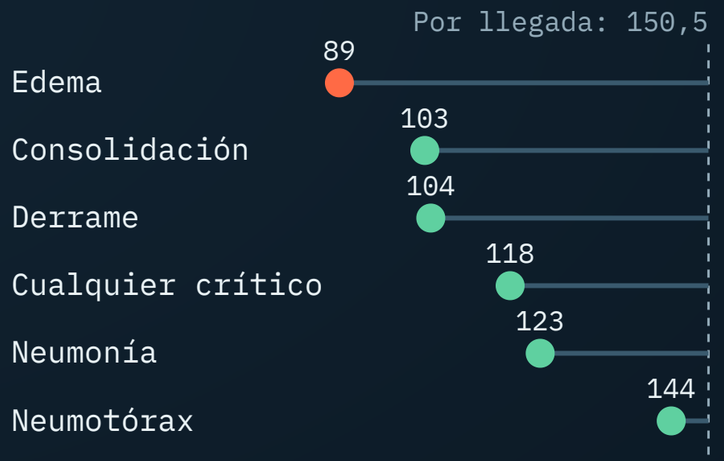
Predicción 7,1× · Medido con los mismos hilos 7,3×
Medido, CPU libre para decodificar 8,3× · Solapado 10,3×

SE ORDENA LA COLA



214 → 71

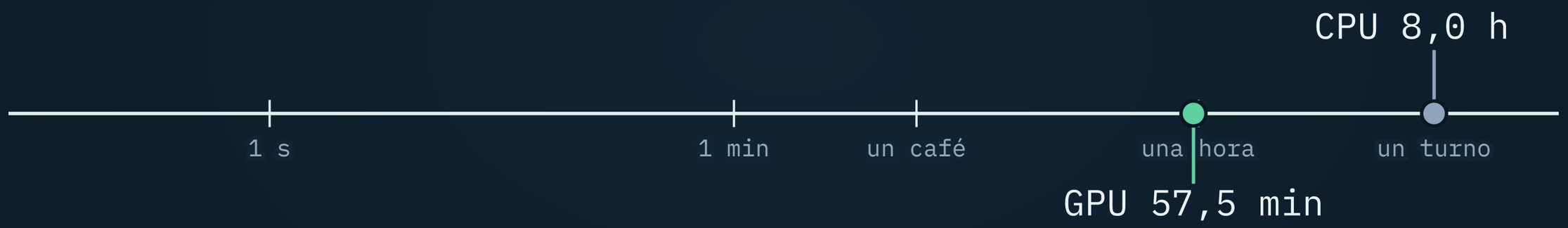
Doña Marta se lee 7 h 09 min antes.



Prioriza, no diagnostica. El radiólogo sigue leyendo todo. AUC de 0,61 a 0,77 · 5 000 colas simuladas

UN TURNO CONTRA UNA HORA

Escuintla, un día · 197 | IGSS, un día · 2 455 | NIH completo · 112 120 | **IGSS, un año · 895 987**



895 987 placas

Volver a puntuar todo el archivo cada vez que mejore el modelo

Recuperar en minutos un atraso tras una caída del sistema

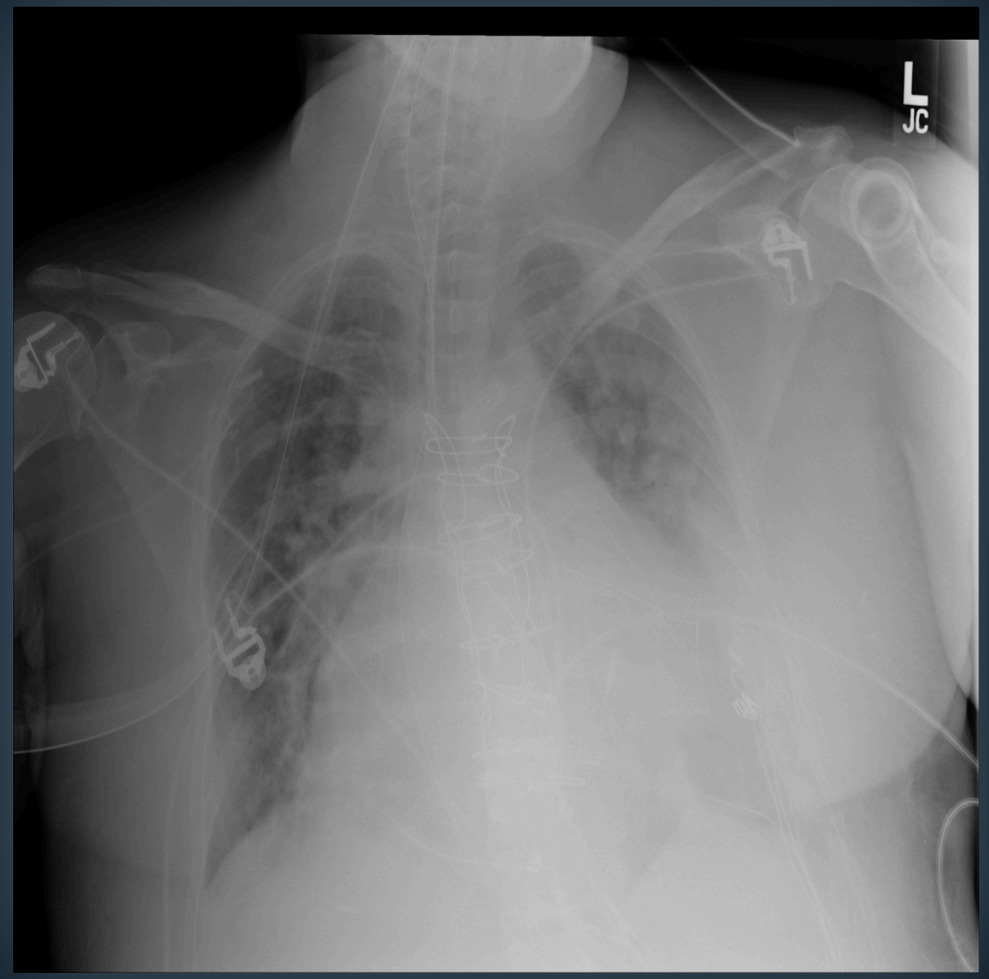
Usar modelos unas 8 veces más costosos en el mismo tiempo

LA MISMA PLACA.
7 HORAS ANTES.

214 → 71
posición en la cola

8,3×
24,1 min a 2,9 min

2,3 %
se fue en copiar



github.com/ecarcamo/Second-CUDA-Challenge

Diego López · Nicolás Concuá · Esteban Cárcamo

FUENTES Y REPRODUCIBILIDAD

Wang et al. (2017). ChestX-ray8. CVPR, pp. 3462–3471. Datos: NIH Clinical Center, nihcc.app.box.com/v/ChestXray-NIHCC.

Cohen et al. (2022). TorchXRayVision. PMLR 172, 231–249.

Huang et al. (2017). DenseNet. CVPR, 2261–2269.

Paszke et al. (2019). PyTorch. NeurIPS 32, 8024–8035.

Okuta et al. (2017). CuPy. LearningSys, NIPS.

Harris et al. (2020). NumPy. Nature 585, 357–362.

Bradski (2000). OpenCV. Dr. Dobb's Journal.

IGSS (2025). Boletín estadístico: prestaciones en salud 2024, p. 56.

IGSS (2026). Hospital del IGSS en Escuintla, nota del 8 de abril.

Annarumma et al. (2019). Radiology 291(1), 196–202.

Cosentini et al. (2009). Intensive Care Medicine 35(2), 299–305.

Fuentes tipográficas: Big Shoulders Display, Hanken Grotesk e IBM Plex Mono, Google Fonts.

Equipo: Intel i9-13980HX y NVIDIA RTX 4050 Laptop de 6 GB. Mediana de 5 corridas tras 1 de calentamiento.

Código, datos y pasos para repetirlo: github.com/ecarcamo/Second-CUDA-Challenge.

FRACCIÓN SECUENCIAL Y PARALELIZABLE

Secuencial es lo que la GPU no acelera: leer y decodificar se quedan en la CPU en las dos versiones. Se mide sobre T_{CPU} porque Amdahl parte del programa original.

ETAPA EN LA VERSIÓN CPU	TIEMPO	% PARTE	
Lectura	19,1 s	1,3	secuencial
Decodificación	53,5 s	3,7	secuencial
Preprocesado	79,2 s	5,5	paralelizable
Inferencia	1 293,7 s	89,5	paralelizable
Puntaje y orden	0,1 s	0,0	paralelizable
T_{CPU}	**1 445,8 s**	**100**	

```
s = (lectura + decodificación) / T_CPU
  = (19,1 + 53,5) / 1 445,8 = 72,6 / 1 445,8
  = 0,0502 → 5,0 %
p = 1 − s = 0,9498 → 95,0 %

Techo: 1 / s = 1 / 0,0502 = 19,9×
k = inferencia CPU / GPU = 1 293,7 / 122,8 = 10,5
Predicción: S = 1 / (s + p / k)
  = 1 / (0,0502 + 0,9498 / 10,5)
  = 1 / 0,1407 = 7,1× · medido con 8 hilos: 7,3×
```

En la versión GPU las mismas etapas secuenciales pesan 26,6 % de T_{GPU}: no se volvieron lentas, el resto se encogió. Las etapas suman 0,2 s menos que el total; esa diferencia queda dentro de p.

¿DÓNDE APLICAMOS CUDA?

Leer PNG	Decodificar	1 · Copiar uint8	2 · Kernel propio	3 · DenseNet-121	4 · Puntaje y orden
CPU	CPU, hilos	memoria fijada, PCIe	CuPy, 3,2 M hilos	cuDNN, FP32	en la GPU

1 Copia por PCIe

```
host = torch.empty(shape, dtype=torch.uint8,
                   pin_memory=True)
device.copy_(host, non_blocking=True)
```

Memoria fijada: la GPU copia directo con DMA, en enteros de 1 byte. **3,96 s, 11,9 GB/s, 2,3 %.**

gpu/run.py

2 Kernel propio en CuPy

```
module = cp.RawModule(code=SOURCE)
kernel((14, 14, n), (16, 16), args)
```

Recorta, reduce a 224 × 224 y normaliza: un hilo por píxel de salida. **0,36 s, 219× frente a la CPU.**

gpu/preprocess.py

3 Red neuronal en cuDNN

```
model = model.cuda()
cudnn.conv.fp32_precision = "ieee"
out = model(x)
```

DenseNet-121 corre en la GPU en FP32, con TF32 apagado. **122,8 s, 10,5× frente a la CPU.**

gpu/run.py

4 Puntaje y orden en la GPU

```
scores = out[:, cols].amax(dim=1)
order = torch.argsort(-scores)
```

El máximo de los 5 hallazgos y la cola se calculan sin salir de la GPU. Solo vuelven los resultados finales.

gpu/run.py

5 Medición con eventos de CUDA

```
e = torch.cuda.Event(enable_timing=True)
e.record()
inicio.elapsed_time(fin)
```

Cada etapa se mide dentro de la GPU, así la copia sale aparte del resto.

gpu/run.py

6 Solapado CPU y GPU

```
future = producer.submit(produce,
                         siguiente_lote)
```

Dos búferes fijados: la CPU decodifica el lote siguiente mientras la GPU procesa el actual. **10,3×.**

gpu/run.py · --modo solapado